

Assignment 1

Movie Ticket Booking System — Object-Oriented System Design

Course	B.Tech. CSE — Semester 5
Subject	System Design (TCS-504)
Assignment	Assignment 1 — Movie Ticket Booking System
Submission Date	07 September 2026
Language used	C++ (one class per file, no header files)

What this document contains

- A. Requirement analysis (FR + NFR)
- B. Noun–verb analysis table
- C. Relationship table with justification
- D. Class diagram
- E. Sequence diagram — book a ticket and make payment
- F. Modular working code + demo run
- G. SOLID mapping + one thing deliberately not done

A. Requirement Analysis

Functional Requirements (FR)

FR4 — Booking (given standard).

A customer selects one or more seat numbers for a show. If any selected seat is already BOOKED, the whole booking is rejected and no seat changes state. Booking is confirmed only after payment succeeds.

FR6 — Payment (given standard).

Exactly one method (UPI / Card / Cash) per booking. If payment fails, seats are released and booking status becomes FAILED.

FR1 — List movies. The system shall display all movies currently playing, showing at least title, language and duration, when the customer selects “Movies” from the main menu.

FR2 — List shows. For a movie chosen by the customer, the system shall list every show of that movie, each identified by its screen number and start time.

FR3 — Seat layout. For a show chosen by the customer, the system shall display every seat's number, type (SILVER/GOLD/PLATINUM) and current status (AVAILABLE or BOOKED), grouped by seat type.

FR5 — Pricing. The system shall calculate the total price of a booking by summing, for every selected seat, the fixed price of that seat's type (SILVER ₹150, GOLD ₹250, PLATINUM ₹400).

FR7 — Ticket printing. Once a booking is CONFIRMED, the system shall print a ticket showing booking id, movie title, screen number, show time, the list of booked seat numbers, and the total amount.

FR8 — Cancellation. The system shall let a customer cancel a CONFIRMED booking by its booking id; on success every seat linked to that booking reverts to AVAILABLE and the booking status becomes CANCELLED. Cancelling a booking that is not CONFIRMED shall be rejected with a clear error message and no state change.

Non-Functional Requirements (NFR)

NFR1 — Modularity. The system shall keep exactly one class per source file with no shared header files, so changing one class's internals never forces edits to another class's file.

NFR2 — Extensibility (Open/Closed). Adding a new payment method (e.g. NetBanking) shall require only one new class deriving from Payment; no existing class — especially BookingService — shall need to be edited.

NFR3 — Robustness / input validation. The system shall validate every user input (menu choice, movie/show index, seat number, payment choice) and print a clear error message instead of crashing or corrupting booking state on invalid input.

NFR4 — Usability. The console menu shall use short, unambiguous prompts and a consistent layout so a non-technical customer can complete a booking without external instructions.

B. Noun–Verb Analysis

Nouns underlined in the problem statement, and whether each earns its own class. Rejecting a noun is design too — the reason is what matters.

Noun found	Keep as a class?	Reason
Movie	Yes	Has its own data (title, language, duration) and identity.
Seat	Yes	Has a number and a type; exists independently of any show.
“seat layout”	No	A view of a Show's seats, not a thing → becomes Show::printSeatLayout().
Screen	Yes	Has a screen number and owns a fixed set of physical seats.
Cinema	Yes	The theatre itself: a name that owns a set of screens.
Show	Yes	One screening = one Movie on one Screen at one time; has its own identity and seat state.
“AVAILABLE / BOOKED status”	No	A property that changes per show → becomes ShowSeat's status field, not its own class.
Customer	Yes	Has a name and phone number; referenced by bookings.
Booking	Yes	Has an id, seats, amount and a status/lifecycle — a real record with identity.
Payment	Yes (abstract)	A contract every payment method must follow; never instantiated on its own.
UPI / Card / Cash	Yes	Each pays differently → become concrete subclasses of Payment (inheritance).
Ticket	No	A printed representation of a Booking, not stored data → becomes TicketPrinter::printTicket().
Price / amount	No	A computed number, not an entity → becomes PriceCalculator's responsibility.
“booking id”	No	Just an attribute of Booking, not a separate class.
Menu / console	No	Procedural flow, not domain data → stays as main()'s menu loop, not a business class.

Classes and their key methods (from Step C, one responsibility each)

Class	Key methods
Movie	getTitle(), getLanguage(), getDurationMin()
Seat	getSeatNumber(), getType()
Screen	addSeat(), getSeats(), getScreenNumber()
Cinema	addScreen(), addMovie(), listMovies()
Show	findShowSeat(), printSeatLayout(), getShowSeats()
ShowSeat	isAvailable(), book(), release()

Class	Key methods
Customer	getName(), getPhone()
Booking	confirm(), fail(), cancel()
Payment (abstract)	pay(amount) = 0, getMethodName() = 0
UpiPayment / CardPayment / CashPayment	pay(amount) [own implementation]
PriceCalculator	priceFor(type), calculateTotal(seats)
TicketPrinter	printTicket(booking)
BookingService	bookSeats(), makePayment(), cancelBooking(), findBooking()

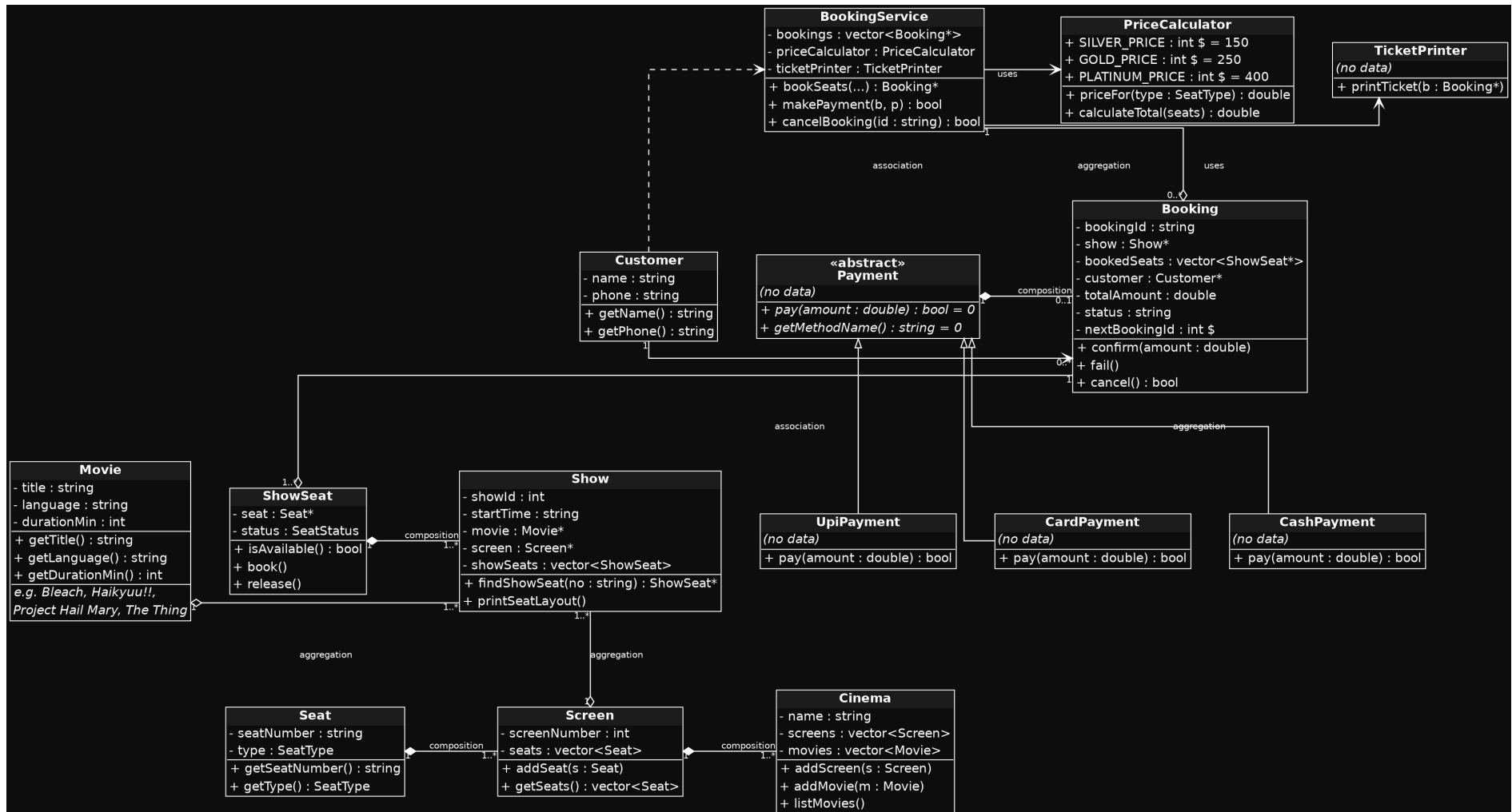
C. Relationships (Step D)

Every relationship below is decided with the lifetime test: if the whole is destroyed, does the part die with it?

Pair	Choice	Justification (lifetime test)
Cinema — Screen	Composition ◆	A Screen number is only meaningful inside one Cinema, and Cinema creates its Screens. Destroy the Cinema and its Screens have nothing left to belong to — they die with it.
Screen — Seat	Composition ◆	A physical seat is bolted into exactly one screen. Destroy the Screen and its Seats go with it; they cannot be moved to another screen.
Show — Movie	Aggregation ◇	(Given standard.) A Show borrows a Movie. Cancel the 6 PM show and the movie “Bleach” still exists and still plays at 9 PM.
Show — Screen	Aggregation ◇	A Show borrows a Screen for its own time slot. Delete that one Show and Screen-1 keeps existing — and keeps hosting other shows.
Show — ShowSeat	Composition ◆	A ShowSeat only means “this seat's status for this show.” Delete the Show and its ShowSeats disappear with it; a new Show gets a brand-new set.
Booking — Customer	Association →	A Customer exists whether or not they have a booking, and a Booking references a customer without owning their lifetime. Neither creates the other.
Booking — ShowSeat	Aggregation ◇	A Booking references existing ShowSeats to mark them booked. Cancel/destroy the Booking and the ShowSeats keep existing — they simply return to AVAILABLE.
Booking — Payment	Composition ◆	The Payment object is created purely to service one Booking's checkout step and has no purpose outside it. It has no life before or after that Booking's payment attempt.
Payment — UpiPayment	Inheritance —▷	UpiPayment IS-A Payment: it inherits the pay() contract and supplies its own implementation.
BookingService — Booking	Aggregation ◇	BookingService creates Bookings, but they must outlive a single method call (looked up later for “My tickets” / cancellation), so they are not destroyed with any one call — the classic “creates but does not own the lifetime” case.

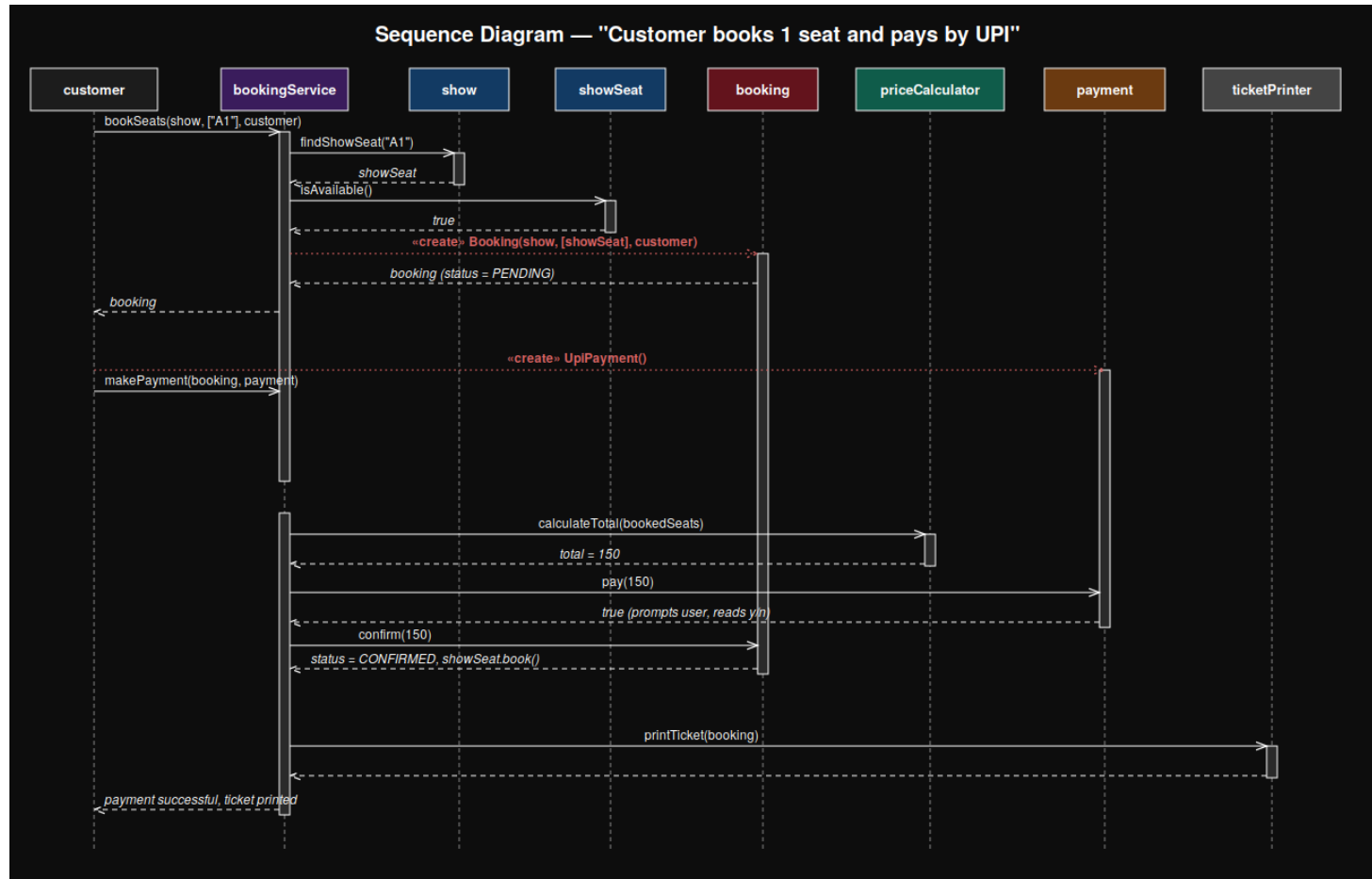
D. Class Diagram (Step E)

All classes from Step C are shown with three compartments (name / attributes / methods), visibility markers (– private, + public, ~ package-default for stateless helper classes), correct UML notation (◆ composition, ◇ aggregation, → association, ▷ inheritance with the Payment hierarchy marked «abstract»), and multiplicities on every structural edge.



E. Sequence Diagram (Step F)

Use case: “customer books 1 seat and pays by UPI,” from the customer choosing a show through the ticket being printed. Lifelines shown: customer, bookingService, show, showSeat, booking, priceCalculator, payment, ticketPrinter. Solid arrows are synchronous calls, dashed arrows are returns, and the dotted red arrows marked «create» show where Booking and Payment objects come into existence.



F. Modular Code + Demo Run (Step G)

Full source is submitted alongside this document as separate .cpp files, one class per file, no header files, matching the required structure:

File	Contains
01_Movie.cpp	Movie entity
02_Seat.cpp	Seat entity + SeatType enum
03_Screen.cpp	Screen entity
04_Cinema.cpp	Cinema entity, F1 (list movies)
05_Show.cpp	Show entity, F2/F3 (shows, seat layout)
06_ShowSeat.cpp	ShowSeat entity + SeatStatus enum
07_Customer.cpp	Customer entity
08_Booking.cpp	Booking entity, F4 support, F8 (cancel)
09_Payment.cpp	Abstract Payment base class
10_PaymentTypes.cpp	UpiPayment, CardPayment, CashPayment (F6)
11_PriceCalculator.cpp	F5 (pricing)
12_TicketPrinter.cpp	F7 (ticket printing)
13_BookingService.cpp	Orchestrator: bookSeats(), makePayment(), cancelBooking()
main.cpp	Console menu, seed data, wiring

Compiled and run with:

```
g++ -std=c++17 -Wall -Wextra -o ticket_booking main.cpp
./ticket_booking
```

Demo run covering all four required edge cases

- 1. Booking a seat that is already BOOKED → rejected, nothing changes
- 2. A failed payment → booking NOT confirmed, seats released
- 3. Cancelling a booking → those seats show AVAILABLE again
- 4. An invalid seat number → a clear message, no crash

```
===== MOVIE TICKET BOOKING =====
```

```
1. Movies  2. Book  3. Cancel  4. My tickets  0. Exit
```

```
Choose:
```

```
[1] Bleach    Japanese      105 min
[2] Haikyuu!! Japanese      85 min
[3] Project Hail MaryEnglish 132 min
[4] The Thing English 109 min
```

```
Choose movie:  [1] Screen-1  06:00 PM
```

```
Choose show:
```

```
SCREEN-1  06:00 PM  |  Bleach
```

SILVER A1[] A2[] A3[] A4[]
GOLD B1[] B2[] B3[]
PLATINUM C1[] C2[]
([] = available [X] = booked)

Seats (e.g. A1,B2): Pay by: 1.UPI 2.Card 3.Cash >

A1 SILVER Rs.150
B2 GOLD Rs.250
TOTAL Rs.400

[UPI] Rs.400 -- confirm payment? (y/n): [UPI] Rs.400 paid successfully

===== TICKET =====

Booking ID : BK1001
Movie : Bleach
Screen : Screen-1 06:00 PM
Seats : A1, B2
Amount : Rs.400 Status: CONFIRMED
=====

1. Movies 2. Book 3. Cancel 4. My tickets 0. Exit

Choose:

[1] Bleach Japanese 105 min
[2] Haikyuu!! Japanese 85 min
[3] Project Hail MaryEnglish 132 min
[4] The Thing English 109 min

Choose movie: [1] Screen-1 06:00 PM

Choose show:

SCREEN-1 06:00 PM | Bleach
SILVER A1[X] A2[] A3[] A4[]
GOLD B1[] B2[X] B3[]
PLATINUM C1[] C2[]
([] = available [X] = booked)

Seats (e.g. A1,B2): Error: seat A1 is already BOOKED. Booking rejected.

1. Movies 2. Book 3. Cancel 4. My tickets 0. Exit

Choose:

[1] Bleach Japanese 105 min
[2] Haikyuu!! Japanese 85 min
[3] Project Hail MaryEnglish 132 min
[4] The Thing English 109 min

Choose movie: [1] Screen-1 06:00 PM

Choose show:

SCREEN-1 06:00 PM | Bleach
SILVER A1[X] A2[] A3[] A4[]
GOLD B1[] B2[X] B3[]
PLATINUM C1[] C2[]
([] = available [X] = booked)

Seats (e.g. A1,B2): Pay by: 1.UPI 2.Card 3.Cash >

B1 GOLD Rs.250
TOTAL Rs.250

[UPI] Rs.250 -- confirm payment? (y/n): Payment FAILED. Booking NOT confirmed; seats remain AVAILABLE.

1. Movies 2. Book 3. Cancel 4. My tickets 0. Exit

Choose:

[1] Bleach Japanese 105 min
[2] Haikyuu!! Japanese 85 min
[3] Project Hail Mary English 132 min
[4] The Thing English 109 min

Choose movie: [1] Screen-1 06:00 PM

Choose show:

SCREEN-1 06:00 PM | Bleach
SILVER A1[X] A2[] A3[] A4[]
GOLD B1[] B2[X] B3[]
PLATINUM C1[] C2[]
([] = available [X] = booked)

Seats (e.g. A1,B2): Error: seat Z9 does not exist on this show.

1. Movies 2. Book 3. Cancel 4. My tickets 0. Exit

Choose: BK1001 Bleach Status: CONFIRMED

BK1002 Bleach Status: FAILED

1. Movies 2. Book 3. Cancel 4. My tickets 0. Exit

Choose: Enter Booking ID to cancel: Booking BK1001 cancelled. Seats are now AVAILABLE.

1. Movies 2. Book 3. Cancel 4. My tickets 0. Exit

Choose: BK1001 Bleach Status: CANCELLED

BK1002 Bleach Status: FAILED

1. Movies 2. Book 3. Cancel 4. My tickets 0. Exit

Choose: Thank you for visiting PVR Dehradun!

G. SOLID Mapping + What We Deliberately Did Not Do

Principle	Where it is applied in this system
S — Single Responsibility	Booking only stores booking data (id, seats, amount, status). It never prints itself (TicketPrinter's job) and never computes its own price (PriceCalculator's job), so a ticket-layout change only touches TicketPrinter, and a pricing-rule change only touches PriceCalculator.
O — Open/Closed	BookingService depends only on the abstract Payment class. Adding NetBankingPayment means writing one new file that extends Payment — BookingService and the existing payment classes need zero edits.
L — Liskov Substitution	Anywhere a Payment* is expected (BookingService::makePayment), any of UpiPayment, CardPayment, or CashPayment can be substituted with no extra setup call and no broken behaviour — all three honour the same pay(amount) → bool contract.
D — Dependency Inversion	BookingService is handed a Payment* chosen by main(), instead of constructing a concrete CardPayment or UpiPayment itself. The high-level orchestration logic never depends on a low-level concrete payment class.

One thing we deliberately did NOT do

We deliberately did not add persistence — there is no file or database save, so Movies, Shows, Bookings and their statuses live only in memory for the duration of one run. Section 2 of the assignment explicitly scopes this as “build exactly this, nothing more,” and does not ask for data to survive a restart, so adding a storage layer would be scope creep rather than a missing requirement.

As a related, smaller cut: Payment does not declare a refund() method (we avoided the Interface Segregation trap of forcing every payment type to support refunds when the assignment's scope never asks for a refund flow at all). If refunds were added later, they would live behind their own small interface implemented only by the payment types that actually support it, rather than bloating the base Payment contract.